Faculté des Sciences de Bizerte

République Tunisienne
Ministère de l'Enseignement Supérieur, de la Recherche Scientifique
et de la Technologie

**Université de Carthage**

Département Informatique

## Mémoire de Projet de Fin d'Études

pour l'obtention du diplôme de licence en sciences de l'informatique

**Intitulé :**

---

# Conception et développement d'une plateforme intelligente de gestion et d'optimisation des services de transport

---

Réalisé au sein de Bee Coders

**Réalisé par :** Ouhichi Ichraf

**Encadré par :**

**Encadrante Académique**

Mme Yosra Bouchoucha

**Encadrant Professionnel**

M. Ahmed Neffati

Année universitaire : 2026–2027

# Remerciements

Cette section sera complétée ultérieurement.

# Table des matières

**Remerciements** **i**

**Introduction Générale** **1**

**1 Contexte Général du Projet** **3**

1.1 Introduction . . . . . 3

1.2 Présentation de l'organisme d'accueil : Bee Coders . . . . . 3

1.2.1 Présentation de l'entreprise Bee Coders . . . . . 3

1.2.2 Valeurs . . . . . 3

1.2.3 Organigramme de Bee Coders . . . . . 4

1.2.4 Informations générales . . . . . 4

1.2.5 Partenaires techniques IT . . . . . 4

1.3 Présentation du projet GoRide . . . . . 5

1.3.1 Cadre du projet . . . . . 5

1.3.2 Problématique . . . . . 5

1.3.3 Étude de l'existant . . . . . 5

1.3.4 Critique de l'existant . . . . . 6

1.3.5 Solution proposée . . . . . 7

1.4 Méthodologie de travail : Scrum (Découpage en Sprints) . . . . . 9

1.4.1 Méthodologie agile Scrum . . . . . 9

1.4.2 Rôles de Scrum . . . . . 10

1.4.3 Artefacts et événements Scrum . . . . . 10

1.5 Langage de modélisation UML . . . . . 10

1.6 Conclusion . . . . . 11

**2 Analyse et Spécification des Besoins 13**

2.1 Introduction . . . . . 13

2.2 Analyse et spécification des besoins . . . . . 13

2.2.1 Spécification des besoins fonctionnels . . . . . 13

2.2.2 Spécification des besoins non fonctionnels . . . . . 15

2.3 Identification et structuration de cas d'utilisation . . . . . 15

2.3.1 Identification des acteurs . . . . . 16

2.3.2 Diagramme de cas d'utilisation global . . . . . 16

2.3.3 Diagramme de classe global . . . . . 17

2.4 Gestion de projet avec SCRUM . . . . . 18

2.4.1 Team et rôles . . . . . 19

2.4.2 Backlog du produit . . . . . 19

2.5 Environnement de travail . . . . . 19

2.5.1 Environnements matériels . . . . . 19

2.5.2 Environnements logiciels . . . . . 20

2.5.3 Technologies et langages utilisées . . . . . 22

2.6 Architecture logicielle . . . . . 24

2.7 Conclusion . . . . . 25

**3 Conception Générale et Architecture 27**

3.1 Introduction . . . . . 27

3.2 Architecture logicielle (Architecture 3-tiers : Frontend, Backend, Base de données) 27

3.3 Environnement technique . . . . . 27

3.3.1 Technologies Backend (Spring Boot) . . . . . 27

3.3.2 Technologies Frontend (React.js / Angular) . . . . . 27

3.3.3 APIs externes (Google Maps API pour la géolocalisation) . . . . . 27

3.4 Conception de la base de données . . . . . 28

3.4.1 Diagramme de classes global . . . . . 28

3.4.2 Logique de Composition Multi-Rôles (Flexibilité des profils utilisateurs) . 28

3.5 Conclusion . . . . . 28

**4 Réalisation - Sprint 1 : Gestion des Accès et des Profils 30**

4.1 Introduction et Backlog du Sprint . . . . . 30

4.2 Analyse et Conception (Diagrammes de séquence d'inscription avec sélection de rôle) . . . . . 30

4.3 Réalisation (Interface de sélection du profil, formulaires dynamiques) . . . . . 30

4.4 Conclusion du Sprint . . . . . 30

**5 Réalisation - Sprint 2 : Recherche et Réservation Intelligente 32**

5.1 Introduction et Backlog du Sprint . . . . . 32

5.2 Analyse et Conception (Logique de suggestion de véhicule selon la capacité bagages) 32

5.3 Réalisation (Interface de recherche, estimation du prix, filtrage des transporteurs) 32

5.4 Conclusion du Sprint . . . . . 32

**6 Réalisation - Sprint 3 : Services Logistiques (Flottes et Entreprises) 34**

6.1 Introduction et Backlog du Sprint . . . . . 34

6.2 Implémentation du Mode Urgence (Algorithme de priorité géographique) . . . . 34

6.3 Interface Partenaire : Gestion du parc automobile et assignation des chauffeurs . 34

6.4 Interface B2B : Solutions pour entreprises (Réservations groupées et budget mensuel) . . . . . 34

6.5 Conclusion du Sprint . . . . . 34

**7 Réalisation - Sprint 4 : Paiement, Notation et Administration 36**

7.1 Introduction et Backlog du Sprint . . . . . 36

7.2 Intégration du module de paiement sécurisé (D17 / Carte Bancaire) . . . . . 36

7.3 Système d'évaluation et de feedback (Calcul de la note moyenne chauffeur) . . . 36

7.4 Dashboard Administrateur (Gestion des comptes et validation des documents KYC) . . . . . 36

7.5 Conclusion du Sprint . . . . . 36

**Conclusion Générale 37**

**Bibliographie et Netographie** **38**

**Annexes** **39**

# Table des figures

1.1 Organigramme de Bee Coders . . . . . 4
1.2 Logos des partenaires techniques de Bee Coders . . . . . 4
1.3 Cycle de la méthodologie Scrum . . . . . 9
1.4 Interactions entre les rôles Scrum . . . . . 10

2.1 Diagramme de cas d'utilisation global de GoRide . . . . . 17
2.2 Diagramme de classe global de GoRide . . . . . 18
2.3 Logo de l'environnement de développement IntelliJ IDEA . . . . . 20
2.4 Logo de XAMPP . . . . . 21
2.5 Logo de Postman . . . . . 21
2.6 Logo de Git et GitHub . . . . . 21
2.7 Logo de Maven . . . . . 21
2.8 Illustration associée à Spring Boot 3.5.x . . . . . 22
2.9 Illustration associée à Spring Data JPA / Hibernate . . . . . 22
2.10 Illustration associée à Spring Data JPA / Hibernate . . . . . 22
2.11 Illustration associée à Angular 16 . . . . . 23
2.12 Illustration associée à TypeScript . . . . . 23
2.13 Illustration associée à Bootstrap 5 . . . . . 23
2.14 Illustration associée à MySQL 8.0 . . . . . 24
2.15 Illustration associée à JWT . . . . . 24
2.16 Schéma illustratif de l'architecture MVC . . . . . 25

# Introduction générale

Le secteur du transport et de la mobilité connaît aujourd'hui une mutation profonde, portée par l'émergence de solutions numériques qui redéfinissent les standards de confort et d'efficacité. En Tunisie, le déploiement de plateformes intelligentes est devenu un levier essentiel pour répondre aux besoins croissants de déplacement, qu'ils soient personnels ou professionnels.

C'est dans cette optique que le projet GoRide a été conçu au sein de l'organisme d'accueil Bee Coders. Ce projet ne se limite pas à une simple mise en relation, car on y intègre une réponse précise à des problématiques logistiques souvent délaissées, telles que la gestion optimisée des bagages et la diversité des profils d'utilisateurs.

Une attention particulière est portée à la flexibilité du système. Grâce à une architecture multi-rôles, on permet à un même utilisateur d'évoluer de manière fluide entre les statuts de client, de chauffeur ou de propriétaire de flotte. Par ailleurs, la sécurité et la réactivité sont renforcées par l'implémentation d'un mode urgence et d'un système de tarification transparente.

Pour garantir la réussite de cette solution, la méthodologie agile Scrum a été adoptée. Le travail a été segmenté en plusieurs sprints itératifs, permettant une validation progressive des fonctionnalités. Le présent rapport est structuré de la manière suivante :

Dans le premier chapitre, le contexte général du projet est exposé et l'organisme d'accueil est présenté.

Dans le deuxième chapitre, on procède à l'analyse et à la spécification des besoins fonctionnels et non fonctionnels.

Le troisième chapitre est consacré à la conception générale, où l'architecture logicielle et la modélisation de la base de données sont détaillées.

Enfin, la réalisation technique est décrite à travers les différents sprints, allant de la gestion des accès à l'intégration des modules de paiement et d'administration.

# Chapitre I

## Contexte général du projet

---

### Plan

1.1 Présentation de l'organisme d'accueil . . . . . 3

1.2 Présentation du projet GoRide . . . . . 5

1.3 Méthodologie adoptée . . . . . 9

1.4 Langage de modélisation (UML) . . . . . 10

---

# Chapitre 1

# Contexte général du projet

## 1.1 Introduction

Dans ce premier chapitre, une présentation de l'organisme d'accueil, Bee Coders, ainsi que du projet GoRide est réalisée. Ensuite, la problématique traitée est exposée afin de justifier la solution proposée. Par la suite, la méthodologie de travail ainsi que le langage de modélisation utilisés sont détaillés.

## 1.2 Présentation de l'organisme d'accueil

### 1.2.1 Présentation de l'entreprise Bee Coders

Au cours du stage de fin d'études, on a été accueilli au sein de Bee Coders, une entreprise de services numériques spécialisée dans le développement de solutions digitales telles que des sites web,des applications mobiles, et des logiciels. Ces solutions sont adaptées aux besoins des entreprises des start-up ,l'entreprise proposant également un accompagnement dans la réalisation de projets. Bee Coders propose des formations sur la plateforme 9antra.tn,permettant aux étudiants et passionnés de de former,d'aquérir de nouvelles compétences et d'obtenir un certificat facilitant leur insertion professionnelle.

### 1.2.2 Valeurs

L'identité de Bee Coders repose sur trois piliers fondamentaux qui ont directement influencé notre manière d'aborder ce stage : La **flexibilité** est perçue comme un atout majeur pour s'adapter aux évolutions technologiques et aux exigences changeantes des clients. L'**accompagnement** occupe également une place centrale, car on considère que la réussite d'un projet passe par un soutien continu et une formation solide aux nouveaux outils numériques. Enfin, la **qualité** se traduit par la volonté de l'entreprise de livrer des produits finis performants, en parfaite adéquation avec les standards actuels du marché de l'IT.

### 1.2.3 Organigramme de Bee Coders

Pour mieux comprendre la structure au sein de laquelle on a évolué, l'organisation interne est illustrée par la figure suivante :

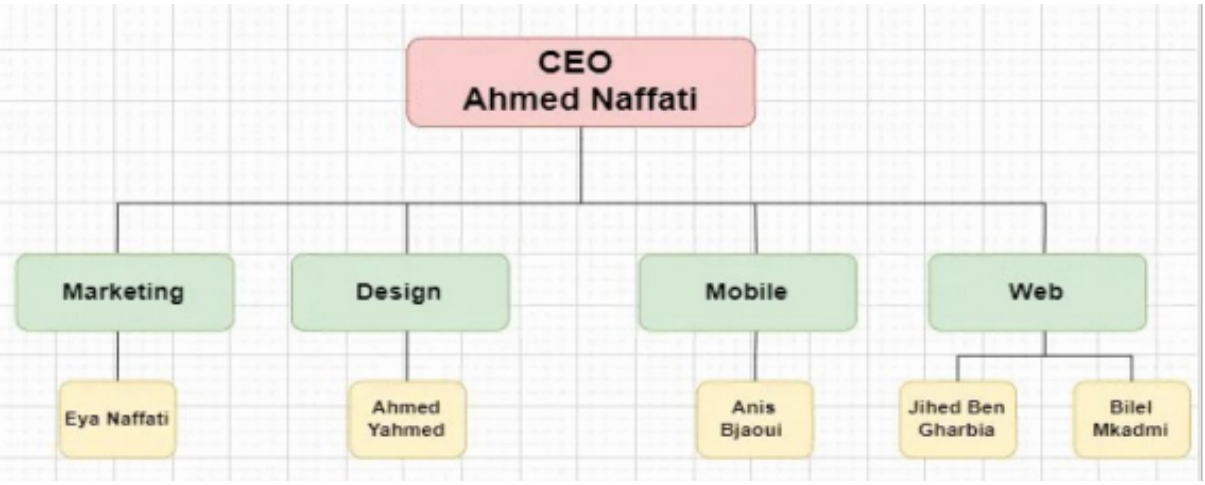


FIGURE 1.1 – Organigramme de Bee Coders

### 1.2.4 Informations générales

Les informations générales relatives à l'entreprise sont les suivantes : le **nom de l'entreprise** est Bee Coders, sa **date de création** remonte à 2020, son **secteur d'activité** concerne les activités informatiques, son **siège** est situé au Parc Technologique El Ghazela à Ariana 208, son **téléphone** est le +216 58 840 06 et son **adresse e-mail** est admin@beecoders.tn.

### 1.2.5 Partenaires techniques IT

Bee Coders travaille avec des partenaires fiables et reconnus sur le marché et intervient comme revendeur officiel de certaines solutions technologiques.

FIGURE 1.2 – Logos des partenaires techniques de Bee Coders

## 1.3 Présentation du projet GoRide

### 1.3.1 Cadre du projet

Le projet GoRide s'inscrit dans une volonté de moderniser la mobilité urbaine à travers le développement d'une plateforme d'intermédiation intelligente. L'objectif principal est d'assurer une mise en relation fluide entre des utilisateurs en quête d'un moyen de transport adapté à leurs exigences (les clients) et des prestataires souhaitant rentabiliser l'usage de leur véhicule (les chauffeurs ou propriétaires).

Dans cette optique, on a conçu une solution qui dépasse le simple cadre du transport standard. On y intègre des outils permettant de personnaliser chaque trajet selon les préférences de l'usager, tout en offrant des leviers concrets pour optimiser les coûts de déplacement. Cette plateforme est ainsi pensée comme un écosystème dynamique où la praticité technologique rencontre les principes de l'économie collaborative, garantissant une expérience à la fois moderne, transparente et accessible à tous.

### 1.3.2 Problématique

Malgré l'abondance des offres de transport, on constate que le processus de mise en relation reste souvent fragmenté et peu adapté aux besoins spécifiques des usagers. La question centrale de ce projet est donc la suivante : Comment peut-on optimiser les services de transport à travers une plateforme intelligente, afin d'offrir une expérience plus pratique, sécurisée et confortable, tout en garantissant une réduction des coûts et une efficacité opérationnelle accrue pour tous les acteurs ?

### 1.3.3 Étude de l'existant

Avant de procéder à la conception de la solution proposée, on a jugé indispensable de mener une analyse approfondie des systèmes déjà en place. Cette étape permet non seulement de cerner les attentes non satisfaites des utilisateurs, mais aussi d'identifier les limites technologiques des solutions existantes, qu'elles soient traditionnelles ou numériques.

#### Solutions traditionnelles

Les moyens de transport classiques, tels que les taxis individuels et collectifs, demeurent des piliers de la mobilité quotidienne en Tunisie. Bien qu'ils soient facilement accessibles, on y rencontre des obstacles récurrents : difficulté de trouver un véhicule durant les heures de pointe, manque de visibilité sur les tarifs et impossibilité de planifier une course à l'avance. Parallèlement, le covoiturage informel s'est largement développé, notamment via les réseaux sociaux. Si cette pratique permet de réduire les coûts, on y déplore une absence de cadre sécurisé, une organisation précaire et une fiabilité des informations très variable, ce qui freine souvent son

adoption à grande échelle.

**Solutions numériques nationales**

L'évolution technologique a favorisé l'émergence de plateformes locales telles que Bolt, inDrive ou Yassir. Ces applications ont marqué un tournant en introduisant la réservation instantanée, la géolocalisation et l'estimation préalable des prix. Cependant, lors de notre analyse, on a relevé que ces outils restent fortement tributaires de la qualité de la connexion Internet et que leur couverture géographique demeure inégale. De plus, on remarque qu'elles ne répondent pas toujours aux besoins logistiques spécifiques, comme la prise en charge de bagages volumineux ou la personnalisation poussée des trajets.

**Solutions numériques internationales**

À l'échelle mondiale, des géants comme Uber ou BlaBlaCar dominent le marché grâce à des infrastructures performantes. On y apprécie la fluidité de l'expérience utilisateur et le suivi rigoureux des prestations. Toutefois, ces solutions présentent des limites économiques, avec des coûts parfois élevés pour le marché local, ainsi qu'une certaine rigidité dans le traitement des besoins spécifiques liés aux flottes d'entreprises ou aux services "sur-mesure"

### 1.3.4 Critique de l'existant

Bien que le paysage du transport, tant au niveau national qu'international, se soit enrichi de nombreuses solutions, on constate que les systèmes actuels peinent encore à satisfaire pleinement les exigences de rapidité, de transparence et de personnalisation. À travers notre analyse, on a pu mettre en évidence trois axes de progression majeurs :

**Une organisation traditionnelle dépassée**

On remarque tout d'abord que les méthodes classiques, comme les taxis individuels ou le co-voiturage informel, ne répondent plus aux standards de la digitalisation. Ces systèmes souffrent d'un manque de structuration et d'une traçabilité limitée, rendant la comparaison des offres difficile pour l'utilisateur. L'imprévisibilité des tarifs et la disponibilité aléatoire des conducteurs créent une expérience souvent frustrante et peu fiable.

**Une rigidité des plateformes numériques actuelles**

Bien que des acteurs comme Bolt, Yassir ou Uber aient révolutionné l'accès au transport, on a relevé des limites structurelles importantes. La majorité de ces applications restent cantonnées à des usages standards. On déplore notamment :

**Un cloisonnement des rôles :** L'utilisateur est souvent enfermé dans un statut unique. Il n'existe pas de véritable écosystème permettant de basculer avec agilité entre le rôle de client, de chauffeur indépendant ou de gestionnaire de flotte. **Une gestion logistique limitée :** Les

besoins spécifiques, tels que le transport de bagages volumineux ou la sélection d'un véhicule selon des critères précis de confort et de capacité, sont rarement pris en compte de manière optimale. **Une dépendance critique au réseau :** La nécessité d'une connexion Internet permanente constitue un frein majeur. En zone de faible couverture, le service devient inaccessible, ce qui impacte la stabilité de l'expérience utilisateur.

**Un manque de personnalisation et de diversité**

Enfin, on constate que les solutions existantes ignorent encore certains besoins émergents de l'économie collaborative. On ne permet pas toujours aux utilisateurs de valoriser leur propre véhicule de manière flexible, ni de bénéficier de services sur-mesure (assistance spécifique, équipements pour enfants, etc.).

Face à ces insuffisances, on a estimé indispensable de concevoir une solution plus résiliente et plus polyvalente. C'est dans cette optique que GoRide a été développée, avec l'ambition de proposer une plateforme intelligente capable de s'adapter aux imprévus du réseau tout en offrant une liberté totale dans le choix des services et des rôles.

### 1.3.5 Solution proposée

Afin de remédier aux limites identifiées dans les solutions existantes, on propose la mise en place d'une plateforme intelligente et centralisée dédiée à la gestion et à la réservation de services de transport. Cette solution est conçue pour accroître la transparence, la flexibilité et l'efficacité des services, tout en s'adaptant précisément aux exigences des utilisateurs.

L'approche adoptée repose sur les principes de l'économie collaborative. Le système est développé de manière à permettre non seulement la réservation de véhicules, mais aussi la comparaison des offres et la personnalisation complète des trajets (type de véhicule, capacité de chargement, prise en charge des bagages). On a opté pour une architecture web moderne et évolutive, garantissant une interaction fluide en temps réel entre les passagers et les prestataires.

**Interface utilisateur intuitive et centralisée**

Pour faciliter le parcours de l'utilisateur, une interface simple et ergonomique est mise en place. On a veillé à ce que la recherche et la réservation soient centralisées afin d'éviter la dispersion des informations :

Les offres disponibles sont regroupées au sein d'un tableau de bord unique. Des filtres multicritères (prix, disponibilité, type de véhicule) sont mis à disposition. Les détails de chaque service sont présentés de façon structurée pour une lecture rapide. L'accessibilité est assurée par une interface réactive (responsive), adaptée aux supports web et mobiles.

**Système intelligent de gestion et de comparaison**

Un module d'intelligence logicielle est intégré pour automatiser l'analyse des offres. Grâce à cet outil, une comparaison automatique des services est effectuée selon les paramètres saisis, un classement pertinent est généré en fonction des préférences de l'utilisateur et des suggestions de trajets optimisés sont proposées par le système. La prise de décision est ainsi simplifiée et le processus de sélection devient plus transparent.

**Optimisation par l'économie collaborative et le partage de frais**

Afin de renforcer la flexibilité du système et de réduire les coûts pour les usagers, une fonctionnalité de partage de ressources et de trajets est intégrée. On permet ainsi aux utilisateurs de devenir des acteurs actifs du réseau tout en réalisant des économies significatives. La mutualisation des trajets prend la forme d'une fonctionnalité de réservation groupée permettant à plusieurs passagers de partager un même itinéraire. La réduction des coûts est assurée par une répartition automatique des frais de transport entre les occupants, tandis que l'exploitation des ressources privées rend possible la mise à disposition de véhicules personnels par les utilisateurs, transformant chaque conducteur potentiel en prestataire de service. En parallèle, l'intermédiation directe établit une relation immédiate entre conducteurs et passagers en supprimant les intermédiaires coûteux, et la gestion en temps réel permet de suivre instantanément les places disponibles ainsi que la confirmation des réservations partagées. On cherche ainsi à optimiser l'occupation des véhicules tout en proposant une alternative de transport plus abordable et durable.

Une approche collaborative du transport est ainsi mise en œuvre.

**Gestion des besoins spécifiques des utilisateurs**

L'expérience globale est améliorée par une personnalisation poussée des services. On a intégré des options modulables pour répondre à chaque profil, notamment par la prise en charge spécifique du transport de bagages volumineux et par l'assurance d'une mise en relation directe entre conducteurs et passagers. Des options de confort (sièges enfants, besoins particuliers) sont proposées lors de la réservation. Le choix du véhicule est laissé à la discrétion de l'utilisateur selon ses besoins de capacité.

**Architecture évolutive et sécurisée**

Sur le plan technique, une structure robuste et modulaire est adoptée. On garantit ainsi la fiabilité et la pérennité du système : La protection des données personnelles et des réservations est assurée par des protocoles sécurisés. LL'historique des transactions est géré de façon structurée pour un suivi optimal. Grâce à la modularité de l'architecture, l'ajout futur de nouvelles fonctionnalités est facilité.

**Conclusion de la solution**

En résumé, la solution proposée se veut à la fois centralisée, intelligente et flexible. En combinant personnalisation et économie collaborative, on parvient à transformer la gestion du transport en un service fluide, transparent et parfaitement adapté aux attentes du marché actuel.

## 1.4 Méthodologie adoptée

Dans le cadre de ce projet, une méthodologie Agile a été adoptée afin de garantir une plus grande flexibilité et une adaptation continue aux besoins des utilisateurs. Par cette approche, on a pu développer le système de manière progressive, tout en veillant à ce que les retours des utilisateurs soient intégrés à chaque étape du cycle de vie.

### 1.4.1 Méthodologie agile Scrum

La méthodologie Agile est basée sur un développement itératif et incrémental, permettant la livraison progressive des fonctionnalités. Elle favorise la collaboration entre les différents acteurs du projet,

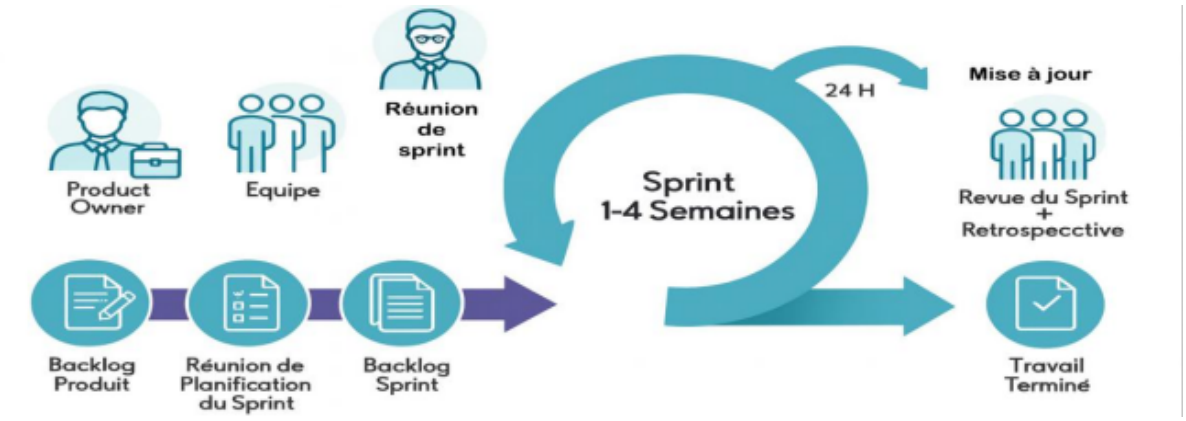


FIGURE 1.3 – Cycle de la méthodologie Scrum

Le processus Agile repose sur un cycle itératif permettant une amélioration continue du produit, comme illustré dans la figure suivante.

Scrum : Parmi les différentes méthodes Agile, Scrum a été retenue pour la réalisation de ce projet. Elle permet d'organiser le travail en cycles appelés sprints, durant lesquels un ensemble de fonctionnalités est développé et livré. Chaque sprint aboutit à un incrément fonctionnel du produit, permettant ainsi une validation progressive et une meilleure qualité du système.*sprints*

### 1.4.2 Rôles de Scrum

Au sein de l'organisation Scrum, plusieurs rôles ont été définis afin d'assurer le bon déroulement du projet : Le **Product Owner** s'est vu confier la responsabilité de définir les besoins et de gérer le Product Backlog, tout en veillant à la priorisation des fonctionnalités en fonction des attentes métiers. Le **Scrum Master**, pour sa part, garantit le respect des principes Scrum, facilite la communication au sein de l'équipe et s'assure que les obstacles rencontrés sont résolus rapidement. Quant à l'**équipe de développement**, elle prend en charge la conception, le développement et les tests dans une logique collaborative afin de livrer un produit fonctionnel à la fin de chaque itération.

Les différents rôles de la méthodologie Scrum et leurs interactions sont illustrés dans la figure suivante.

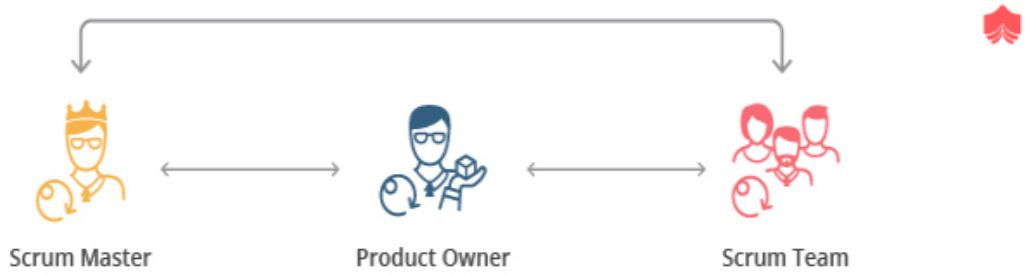


FIGURE 1.4 – Interactions entre les rôles Scrum

### 1.4.3 Artefacts et événements Scrum

Concernant les **artefacts**, le **Product Backlog** correspond à une liste exhaustive des fonctionnalités à développer, le **Sprint Backlog** regroupe les tâches sélectionnées pour un sprint donné, tandis que l'**incrément** représente une version fonctionnelle et testée du produit.

S'agissant des **événements**, le **Sprint Planning** permet de planifier les tâches à réaliser durant le sprint, le **Daily Scrum** assure un suivi quotidien afin de coordonner l'équipe et de détecter rapidement les blocages, la **Sprint Review** sert à présenter le travail réalisé pour valider les fonctionnalités développées, et la **Sprint Retrospective** favorise une réflexion collective destinée à améliorer en continu l'organisation et les méthodes de travail.

## 1.5 Langage de modélisation UML

Afin d'analyser efficacement les besoins du système et de garantir une conception cohérente, on a opté pour l'utilisation du langage UML (Unified Modeling Language). Ce langage est largement privilégié dans les processus itératifs, car il permet une représentation visuelle des différentes composantes du système. Grâce à cette modélisation, la communication entre les

différents acteurs du projet est facilitée. Dans UML, deux catégories principales de diagrammes sont distinguées : les diagrammes de structure, par lesquels la structure statique du système est représentée. les diagrammes de comportement, qui sont utilisés pour décrire le comportement dynamique de la solution.

Plusieurs diagrammes UML ont été élaborés pour illustrer le fonctionnement de la plateforme. Le diagramme de cas d'utilisation est exploité pour représenter les interactions entre les acteurs (client, administrateur) et les services offerts. Les diagrammes de séquence sont utilisés pour décrire de manière chronologique les échanges entre les composants, notamment lors du processus de réservation. Enfin, le diagramme de classes détaille la structure statique en mettant en évidence les classes (Utilisateur, Véhicule, Réservation) et les relations qui les lient.

## 1.6 Conclusion

Dans ce premier chapitre, les objectifs clés du projet ont été identifiés et les principaux défis ont été mis en évidence. Une solution innovante a été proposée pour optimiser la gestion des services de transport. On a constaté que l'usage de méthodes traditionnelles engendrait des problèmes de coordination ; c'est pourquoi la plateforme a été conçue en adoptant les méthodologies Scrum et UML pour encadrer rigoureusement le développement.

# Chapitre II

## Analyse et spécification des besoins

---

### Plan


II.1 Introduction . . . . . 13

II.2 Analyse et spécification des besoins . . . . . 13

II.3 Identification et structuration de cas d'utilisation . . . . . 15

II.4 Gestion de projet avec SCRUM . . . . . 18

II.5 Environnement de travail . . . . . 19

II.6 Architecture logicielle . . . . . 24

II.7 Conclusion . . . . . 25


---

# Chapitre 2

# Analyse et spécification des besoins

## 2.1 Introduction

Dans ce chapitre, on s'intéresse à l'analyse détaillée des besoins liés à la plateforme GoRide. L'objectif est d'identifier précisément les exigences fonctionnelles, de structurer les cas d'utilisation et de mettre en place un cadre méthodologique cohérent pour la suite du projet.

Cette étape joue un rôle central, car elle permet de transformer les attentes exprimées en spécifications claires et exploitables dans les phases de conception et de réalisation. Elle sert également à organiser les différents volets du projet, qu'ils soient fonctionnels, techniques ou organisationnels.

## 2.2 Analyse et spécification des besoins

Cette section détaille les exigences que la plateforme GoRide doit satisfaire. On distingue classiquement les besoins fonctionnels, qui décrivent le comportement attendu du système, des besoins non fonctionnels, qui portent sur les critères de qualité et les contraintes techniques.

### 2.2.1 Spécification des besoins fonctionnels

Les besoins fonctionnels traduisent l'ensemble des services que le système doit offrir aux différents acteurs. Dans le cadre de GoRide, ils ont été organisés en plusieurs blocs cohérents afin de refléter les priorités métier de la plateforme.

**A. Gestion des accès et de la sécurité des profils**

Le système doit assurer une gestion granulaire des utilisateurs. On a défini les fonctionnalités suivantes : L'**auto-enregistrement (Self-Registration)** permet à tout nouvel utilisateur de créer un compte, tandis que le formulaire d'inscription valide dynamiquement les données, notamment l'unicité de l'email, le format du numéro de téléphone et la force du mot de passe à

travers des expressions régulières. L'**authentification multi-rôles** permet ensuite au système de reconnaître l'utilisateur et de lui attribuer les droits correspondants (Client, Driver, Fleet Owner, etc.) ; elle repose sur le standard JWT, assurant une communication sécurisée et « stateless » entre le frontend et le backend. Enfin, l'**administration des profils** donne à chaque utilisateur un espace personnel pour modifier ses informations, comme le nom, le prénom, les coordonnées et le mot de passe, ce qui garantit l'exactitude des données de contact essentielles pour la coordination des trajets.

**B. Gestion de la flotte et logistique (Fleet Management)**

C'est le module « nerf de la guerre » pour les partenaires logistiques. On a identifié les besoins suivants : **Inventaire technique des véhicules :** On offre aux propriétaires de flottes la possibilité de déclarer leurs véhicules. Chaque fiche véhicule doit contenir des informations précises : immatriculation, marque, modèle, type de carburant. **Configuration des services à bord :** Le système doit permettre de spécifier les options disponibles pour chaque véhicule (présence du WiFi, climatisation, siège bébé, capacité maximale de bagages). Ces métadonnées sont cruciales pour l'expérience client. **Assignation dynamique Chauffeur–Véhicule :** On a implémenté une logique permettant à l'administrateur de lier un chauffeur certifié à un véhicule spécifique. Cette opération garantit une supervision centrale de l'activité logistique et une gestion optimale des disponibilités. **Gestion financière et validation des offres :** Le propriétaire de flotte dispose d'un pouvoir décisionnel sur l'aspect économique de ses véhicules. Il peut accepter ou refuser les propositions tarifaires soumises par les chauffeurs pour l'utilisation de ses voitures. En cas d'acceptation, le système déclenche le transfert des fonds et assure la traçabilité des revenus générés par le parc automobile.

**C. Réservation intelligente et mode urgence**

La réservation constitue le cœur opérationnel de la plateforme. Les besoins suivants ont été retenus : La **recherche multicritère** permet au client de chercher un trajet en précisant sa destination et ses besoins logistiques, notamment le nombre de passagers et le volume de bagages. Le mécanisme d'**estimation et de confirmation** calcule ensuite une estimation du tarif et propose les véhicules correspondants ; une fois le choix validé, une notification est envoyée au transporteur concerné. Par ailleurs, le **mode urgence** répond à un besoin spécifique en permettant de déclencher une demande prioritaire, dans laquelle le système identifie les véhicules les plus proches et les alerte instantanément afin de réduire le temps d'attente au minimum.

**D. Supervision et administration globale**

Afin d'assurer le bon fonctionnement global de la plateforme, l'administrateur doit disposer d'outils de contrôle et de suivi adaptés : Le **tableau de bord de modération** permet à l'administrateur de lister tous les comptes, de valider les inscriptions professionnelles (chauffeurs, entreprises) et de suspendre les comptes en cas de comportement inapproprié. Les **statistiques de santé**, quant à elles, fournissent des indicateurs sur le nombre d'utilisateurs actifs et le volume de réservations traitées.

Ces besoins fonctionnels montrent que GoRide ne se limite pas à la réservation d'un trajet. La plateforme ambitionne d'intégrer, dans un même environnement, la gestion des accès, la supervision logistique, l'optimisation des réservations et l'administration globale du service.

### 2.2.2 Spécification des besoins non fonctionnels

Les besoins non fonctionnels sont tout aussi critiques car ils conditionnent l'acceptation de la plateforme par les utilisateurs et sa robustesse technique.

**Sécurité et Confidentialité (Norme ISO/IEC 25010).** L'**authentification et l'autorisation** reposent sur le mécanisme Role-Based Access Control (RBAC), et l'accès à chaque point de terminaison de l'Application Programming Interface (API) est protégé par Spring Security. Le **hachage des données** est assuré par l'algorithme BCrypt avec un grain de sel (salt) aléatoire, rendant toute tentative de décryptage par force brute inopérante. Enfin, les **échanges sécurisés** garantissent que toutes les communications sont chiffrées et que les jetons JWT sont signés numériquement pour empêcher toute altération.

**Performance et Réactivité.** Le **temps de latence** visé impose que le système réponde aux requêtes de consultation en moins de 150 ms ; pour cela, les requêtes Structured Query Language (SQL) ont été optimisées via la Java Persistence API (JPA) afin d'éviter le problème des « N+1 select ». La **gestion des accès concurrents** exige par ailleurs que l'architecture permette à plusieurs centaines d'utilisateurs de naviguer simultanément sans dégradation notable des performances.

**Disponibilité et Fiabilité.** Le **taux de disponibilité** visé atteint un « uptime » de 99.9%, tandis que le backend Spring Boot est conçu pour être déployé sur un serveur d'application Java robuste (Embedded Tomcat) facilitant la haute disponibilité. La **robustesse** impose également que le système gère gracieusement les erreurs, par exemple en cas de perte de connexion à la base de données, en renvoyant des messages explicites sans exposer la stack technique.

**Ergonomie et Utilisabilité.** Le **Responsive Design** de l'interface développée avec Angular et Bootstrap permet une adaptation automatique aux résolutions des smartphones, tablettes et ordinateurs. En complément, l'**accessibilité** a été prise en compte afin que l'interface reste intuitive, même pour des utilisateurs peu familiers avec les outils numériques.

**Maintenabilité et Évolutivité.** La **modularité** du code, découpé en composants et services réutilisables, permet d'ajouter un nouveau mode de transport ou un nouveau module de paiement sans réécrire le noyau de l'application.

## 2.3 Identification et structuration de cas d'utilisation

La modélisation des cas d'utilisation constitue une étape pivot dans la compréhension des interactions fonctionnelles de la plateforme GoRide. On utilise cette approche pour capturer les exigences du point de vue de l'utilisateur final, en identifiant précisément qui fait quoi au sein du

système. Cette structuration permet non seulement de délimiter le périmètre de l'application, mais aussi de servir de base à la conception détaillée et aux scénarios de tests. Dans ce qui suit, on procède à l'identification des acteurs et à la présentation des diagrammes illustrant les scénarios d'utilisation majeurs.

### 2.3.1 Identification des acteurs

L'analyse des interactions a permis de définir une hiérarchie d'acteurs claire, facilitant la gestion des droits d'accès.

**Acteur Public (Le Visiteur) :** C'est tout utilisateur non identifié. Il a accès à la page d'accueil, aux informations sur les services et aux formulaires de création de compte. **Acteur Pivot (L'Utilisateur Connecté) :** C'est la classe de base. Tout utilisateur authentifié hérite de ses droits (consulter son profil, modifier son mot de passe, recevoir des notifications). **Acteurs Spécialisés (Héritage métier) : Client :** Il utilise la plateforme pour consommer les services de transport. **Chauffeur (Driver) :** Il est le prestataire de service. Il interagit avec les missions qui lui sont assignées. **Propriétaire (Fleet Owner) :** Acteur logistique et économique majeur, il gère ses véhicules. Son objectif est de maximiser la rentabilité de sa flotte en validant les propositions financières des chauffeurs et en suivant ses revenus en temps réel. **Entreprise (Company) :** Représente un compte corporate avec des besoins de facturation centralisée. **Administrateur :** Il possède les droits de supervision sur l'ensemble de la base de données et de la configuration système.

### 2.3.2 Diagramme de cas d'utilisation global

Le diagramme de cas d'utilisation global est l'outil du Unified Modeling Language (UML) qui permet de visualiser le périmètre fonctionnel du projet. Il met en scène les acteurs et les services majeurs. Le diagramme présenté ci-après illustre la cartographie complète des interactions entre les différents acteurs et les fonctionnalités clés de la plateforme GoRide.

FIGURE 2.1 – Diagramme de cas d'utilisation global de GoRide

### 2.3.3 Diagramme de classe global

Le diagramme de cas d'utilisation global est l'outil du Unified Modeling Language (UML) qui permet de visualiser le périmètre fonctionnel du projet. Il met en scène les acteurs et les services majeurs. Le diagramme présenté ci-après illustre la cartographie complète des interactions entre les différents acteurs et les fonctionnalités clés de la plateforme GoRide.

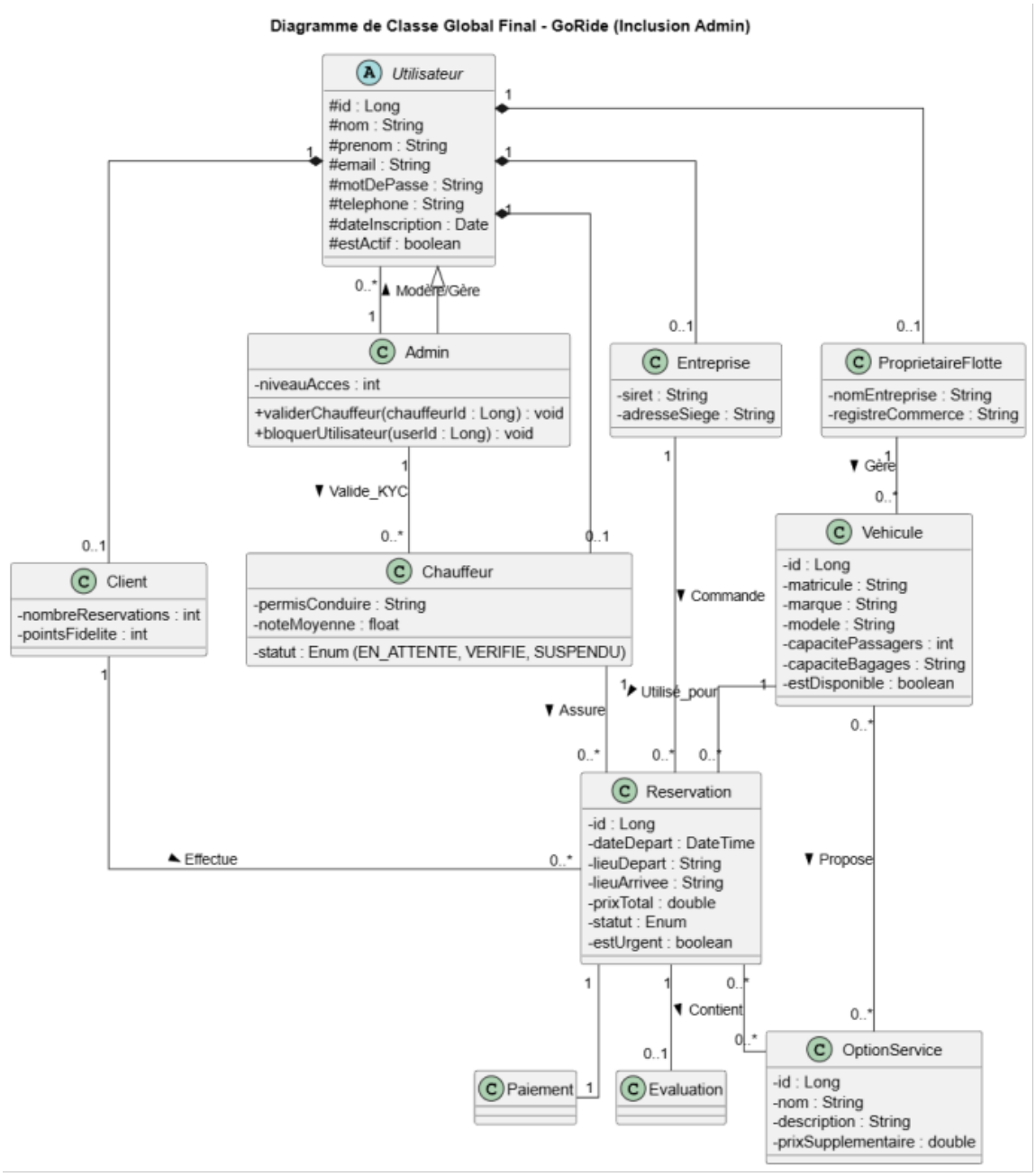


FIGURE 2.2 – Diagramme de classe global de GoRide

## 2.4 Gestion de projet avec SCRUM

Le projet GoRide a été mené selon une approche Agile, en s'appuyant sur le framework Scrum. Ce choix méthodologique a permis de livrer de la valeur de manière incrémentale, tout en garantissant une grande réactivité face aux éventuels ajustements ou changements en cours de développement.

### 2.4.1 Team et rôles

L'organisation de l'équipe a été calquée sur les standards SCRUM :

**Le Product Owner (PO) :** En lien étroit avec Bee Coders, il a défini la vision du produit. Son rôle a été de maintenir le Product Backlog et de s'assurer que chaque fonctionnalité développée apporte une valeur réelle aux futurs utilisateurs. **Le Scrum Master (SM) :** Il a veillé au respect des rituels. On a organisé des réunions quotidiennes (Daily Stand-up) pour synchroniser le travail et identifier les points de blocage techniques, notamment lors de l'intégration du JWT. **La Development Team :** Responsable de la transformation des exigences en code fonctionnel. On a mis en place des revues de code systématiques pour garantir la qualité et la maintenabilité de la stack Java/Angular.

### 2.4.2 Backlog du produit

Le Product Backlog est une liste vivante d'exigences. On a utilisé la technique du « Planning Poker » pour estimer la complexité de chaque User Story (US) en points d'effort.

| ID | User Story (En tant que... je veux... afin de...) | Priorité | Effort (Points) |
|---|---|---|---|
| US01 | En tant qu'utilisateur, je veux m'inscrire pour accéder à la plateforme. | Critique | 5 |
| US02 | En tant qu'utilisateur, je veux m'authentifier pour sécuriser mon espace. | Critique | 3 |
| US03 | En tant que propriétaire, je veux déclarer mes véhicules pour les louer. | Haute | 8 |
| US04 | En tant qu'administrateur, je veux assigner un chauffeur à une voiture. | Haute | 5 |
| US04b | En tant que propriétaire, je veux valider une offre tarifaire d'un chauffeur pour encaisser mes revenus. | Haute | 8 |
| US05 | En tant que client, je veux rechercher un trajet avec mes bagages. | Haute | 8 |
| US06 | En tant que client, je veux activer le mode urgence pour un trajet rapide. | Moyenne | 13 |
| US07 | En tant qu'utilisateur, je veux modifier mes coordonnées personnelles. | Basse | 2 |
| US08 | En tant qu'administrateur, je veux bannir un utilisateur malveillant. | Basse | 5 |
| US09 | En tant qu'entreprise, je veux suivre les factures de mes collaborateurs. | Moyenne | 8 |

Table 2.1: Product Backlog de GoRide

## 2.5 Environnement de travail

Cette section décrit les outils et infrastructures qui ont permis la réalisation concrète du projet.

### 2.5.1 Environnements matériels

Pour mener à bien le développement de la plateforme GoRide, on a utilisé une station de travail haute performance capable de supporter les exigences de compilation et d'exécution simultanée

des environnements backend et frontend. Le travail a été réalisé sur un ordinateur portable aux caractéristiques techniques suivantes :

| **Composant** | **Description** |
|---|---|
| Machine | ASUS TUF Gaming A15. |
| Processeur | AMD Ryzen 7 (Série 4000/5000 haute performance) permettant un traitement multitâche fluide. |
| Mémoire Vive (RAM) | 16 Go de type DDR4, essentiels pour l'exécution fluide de l'IDE (IntelliJ IDEA), de la base de données MySQL et du serveur de développement Angular. |
| Stockage | 512 Go Solid State Drive (SSD) NVMe, garantissant des temps d'accès aux données et de déploiement ultra-rapides. |
| Carte Graphique | NVIDIA GeForce RTX, offrant une puissance de calcul supplémentaire et une gestion optimale de l'affichage multi-écrans. |
| Système d'exploitation | Windows 11 Professionnel. |

Table 2.2: Environnement matériel utilisé pour le développement de GoRide

### 2.5.2 Environnements logiciels

Le choix des logiciels a été dicté par l'efficacité, la fiabilité et le support communautaire. L'application repose sur un environnement d'exécution standard, à savoir la Java Virtual Machine (JVM) pour le backend et le runtime Node.js pour le frontend. Parmi les outils mobilisés, **IntelliJ IDEA** a constitué l'Integrated Development Environment (IDE) central pour le développement de l'intégralité de la stack. On l'a utilisé aussi bien pour le backend (Spring Boot) que pour le frontend (Angular), en profitant de ses capacités avancées de débogage, de complétion de code et d'intégration des outils de build.

Figure 2.3 – Logo de l'environnement de développement IntelliJ IDEA

**XAMPP** a été utilisé pour gérer localement notre serveur de base de données MySQL. Cet ensemble de logiciels offre une interface simplifiée pour l'administration des données et assure une disponibilité constante du service de persistance.

FIGURE 2.4 – Logo de XAMPP

**Postman :** Outil indispensable pour tester et valider les points de terminaison (endpoints) de l'Application Programming Interface (API) REST. Il nous a permis de simuler les requêtes HTTP (GET, POST, PATCH, DELETE) et de vérifier la conformité des réponses JSON avant leur intégration finale.

FIGURE 2.5 – Logo de Postman

**Git & GitHub :** Utilisés pour assurer le versionnage (versioning) du code source et permettre un suivi rigoureux de l'évolution du projet.

FIGURE 2.6 – Logo de Git et GitHub

**Maven :** Gestionnaire de dépendances utilisé pour automatiser la construction et la gestion des bibliothèques du projet Java.

FIGURE 2.7 – Logo de Maven

### 2.5.3 Technologies et langages utilisées

Le projet repose sur une stack technologique moderne, dite « Full-Stack Java/Angular ».

**Backend (Le moteur du système) : Java 17 :** On a choisi cette version pour ses performances accrues et ses nouvelles fonctionnalités syntaxiques, telles que les *Records* et les *Sealed Classes*.

FIGURE 2.8 – Illustration associée à Spring Boot 3.5.x

**Spring Boot 3.5.x :** C'est le framework dominant pour les applications d'entreprise. On profite de l'auto-configuration, de la gestion simplifiée des dépendances et de son intégration avec Spring Security.

FIGURE 2.9 – Illustration associée à Spring Data JPA / Hibernate

**Spring Data JPA / Hibernate :** Ces technologies assurent le mapping entre les objets Java et la base de données relationnelle MySQL, tout en réduisant considérablement le code SQL répétitif.

FIGURE 2.10 – Illustration associée à Spring Data JPA / Hibernate

**Frontend (L'interface utilisateur) : Angular 16 :** Ce framework permet de construire une application modulaire et robuste. On a utilisé les *Signals* et les *Standalone Components* afin d'optimiser les performances de rendu.

FIGURE 2.11 – Illustration associée à Angular 16

**TypeScript :** Le typage fort de TypeScript a permis de détecter de nombreuses erreurs dès la phase de développement, améliorant ainsi la fiabilité du code.

FIGURE 2.12 – Illustration associée à TypeScript

**Bootstrap 5 :** Cette bibliothèque a été utilisée pour assurer une mise en page soignée et *responsive* sans effort de développement excessif.

FIGURE 2.13 – Illustration associée à Bootstrap 5

**Infrastructure et Base de Données : MySQL 8.0 :** Il s'agit d'une base de données relationnelle stable, performante et parfaitement adaptée aux besoins de persistance de GoRide.

FIGURE 2.14 – Illustration associée à MySQL 8.0

**JWT (`io.jsonwebtoken`) :** Cette bibliothèque est utilisée pour la gestion de l'authentification sans session côté serveur.

FIGURE 2.15 – Illustration associée à JWT

## 2.6 Architecture logicielle

Pour concevoir GoRide, le choix s'est porté sur l'architecture Modèle–Vue–Contrôleur (MVC). Il s'agit d'un modèle de conception très courant dans le développement d'applications web. Son principe fondamental est de diviser le projet en trois grandes parties indépendantes : le modèle, la vue et le contrôleur. Cette approche est particulièrement intéressante car elle permet d'avoir un code bien organisé, plus facile à maintenir et à tester.

Dans le contexte de notre plateforme, ces trois blocs se traduisent de la manière suivante.

**Le Modèle.** Il s'occupe de tout ce qui concerne les données et la logique métier, c'est-à-dire les règles de l'application. Concrètement, dans GoRide, ce rôle est assuré par Spring Boot à travers les entités Java Persistence API (JPA), les *Repositories* qui communiquent avec la base MySQL, ainsi que les classes de service. C'est dans cette partie que l'on vérifie, par exemple, les conditions d'assignation d'un véhicule avant de sauvegarder l'information.

**La Vue.** C'est la partie visible par l'utilisateur, autrement dit l'interface graphique. Pour ce projet, toute la vue a été développée avec le framework Angular. Son rôle est de récupérer les informations depuis le modèle et de les présenter de façon claire et *responsive*. Elle intercepte aussi les actions de l'utilisateur, comme le remplissage d'un formulaire ou le clic sur un bouton de réservation, afin de les transmettre au contrôleur.

**Le Contrôleur.** Il sert de chef d'orchestre entre la vue et le modèle. Côté serveur, ce sont nos

*REST Controllers* sous Spring Boot qui remplissent cette mission. Lorsqu'un utilisateur effectue une action depuis l'interface Angular, le contrôleur reçoit la requête HTTP, demande au modèle d'exécuter le traitement nécessaire, puis renvoie le résultat à la vue, généralement sous format JSON. Il gère également les premières vérifications de sécurité, notamment la validation des jetons JWT.

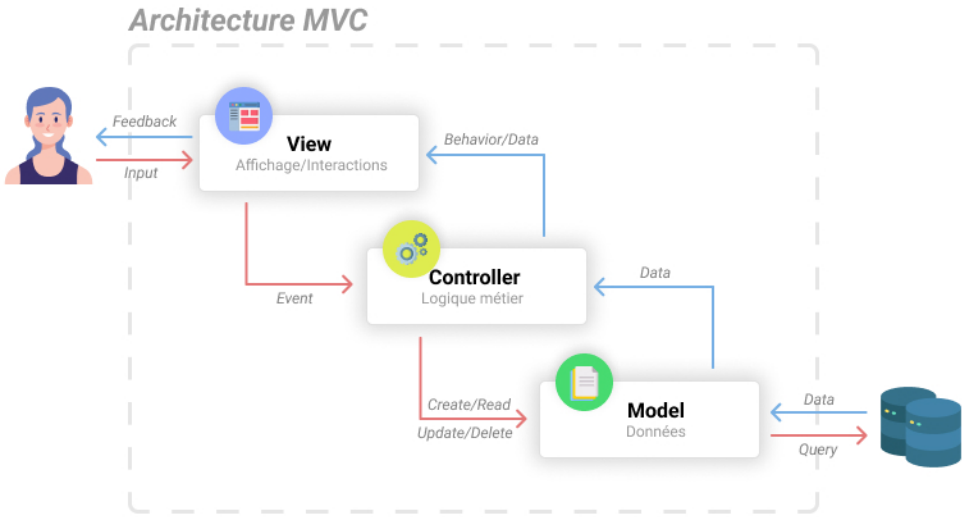


FIGURE 2.16 – Schéma illustratif de l'architecture MVC

## 2.7 Conclusion

En conclusion, ce deuxième chapitre a permis de dresser un portrait exhaustif des spécifications de la plateforme GoRide. L'analyse rigoureuse des besoins fonctionnels, couplée à une modélisation UML précise, nous a offert une vision claire des interactions entre les différents acteurs. Le choix d'une méthodologie Agile SCRUM et d'un environnement technique de pointe (Java 17 / Angular 16) garantit que le développement se fera sur des bases solides et modernes.

On dispose désormais de tous les éléments nécessaires pour aborder sereinement la phase de réalisation technique, où les concepts analytiques ici définis prendront vie à travers le code. Le chapitre suivant sera consacré à la conception détaillée et à l'implémentation des premiers Sprints du projet.

# Chapitre III

## Conception générale et architecture

---

### Plan

3.1 Introduction ..... 27

3.2 Architecture logicielle ..... 27

3.3 Environnement technique ..... 27

3.4 Conception de la base de données ..... 28

3.5 Conclusion ..... 28

---

# Chapitre 3

# Conception générale et architecture

## 3.1 Introduction

Cette section est en cours de rédaction.

## 3.2 Architecture logicielle (Architecture 3-tiers : Frontend, Backend, Base de données)

Cette section est en cours de rédaction.

## 3.3 Environnement technique

Cette section est en cours de rédaction.

### 3.3.1 Technologies Backend (Spring Boot)

Cette sous-section est en cours de rédaction.

### 3.3.2 Technologies Frontend (React.js / Angular)

Cette sous-section est en cours de rédaction.

### 3.3.3 APIs externes (Google Maps API pour la géolocalisation)

Cette sous-section est en cours de rédaction.

## 3.4 Conception de la base de données

Cette section est en cours de rédaction.

### 3.4.1 Diagramme de classes global

Cette sous-section est en cours de rédaction.

### 3.4.2 Logique de Composition Multi-Rôles (Flexibilité des profils utilisateurs)

Cette sous-section est en cours de rédaction.

## 3.5 Conclusion

Cette section est en cours de rédaction.

# Chapitre IV

## Réalisation - Sprint 1 : Gestion des Accès et des Profils

---

### Plan

4.1 Introduction et Backlog du Sprint . . . . . 30

4.2 Analyse et Conception . . . . . 30

4.3 Réalisation . . . . . 30

4.4 Conclusion du Sprint . . . . . 30

---

# Chapitre 4

# Réalisation - Sprint 1 : Gestion des Accès et des Profils

## 4.1 Introduction et Backlog du Sprint

Cette section est en cours de rédaction.

## 4.2 Analyse et Conception (Diagrammes de séquence d'inscription avec sélection de rôle)

Cette section est en cours de rédaction.

## 4.3 Réalisation (Interface de sélection du profil, formulaires dynamiques)

Cette section est en cours de rédaction.

## 4.4 Conclusion du Sprint

Cette section est en cours de rédaction.

# Chapitre V

## Réalisation - Sprint 2 : Recherche et Réservation Intelligente

---

### Plan


5.1 Introduction et Backlog du Sprint ..... 32

5.2 Analyse et Conception ..... 32

5.3 Réalisation ..... 32

5.4 Conclusion du Sprint ..... 32


---

# Chapitre 5

# Réalisation - Sprint 2 : Recherche et Réservation Intelligente

## 5.1 Introduction et Backlog du Sprint

Cette section est en cours de rédaction.

## 5.2 Analyse et Conception (Logique de suggestion de véhicule selon la capacité bagages)

Cette section est en cours de rédaction.

## 5.3 Réalisation (Interface de recherche, estimation du prix, filtrage des transporteurs)

Cette section est en cours de rédaction.

## 5.4 Conclusion du Sprint

Cette section est en cours de rédaction.

# Chapitre VI

## Réalisation - Sprint 3 : Services Logistiques (Flottes et Entreprises)

---

### Plan


6.1 Introduction et Backlog du Sprint ..... 34

6.2 Implémentation du Mode Urgence ..... 34

6.3 Interface Partenaire ..... 34

6.4 Interface B2B ..... 34

6.5 Conclusion du Sprint ..... 34


---

# Chapitre 6

# Réalisation - Sprint 3 : Services Logistiques (Flottes et Entreprises)

## 6.1 Introduction et Backlog du Sprint

Cette section est en cours de rédaction.

## 6.2 Implémentation du Mode Urgence (Algorithme de priorité géographique)

Cette section est en cours de rédaction.

## 6.3 Interface Partenaire : Gestion du parc automobile et assignation des chauffeurs

Cette section est en cours de rédaction.

## 6.4 Interface B2B : Solutions pour entreprises (Réservations groupées et budget mensuel)

Cette section est en cours de rédaction.

## 6.5 Conclusion du Sprint

Cette section est en cours de rédaction.

# Chapitre VII

## Réalisation - Sprint 4 : Paiement, Notation et Administration

---

### Plan


7.1 Introduction et Backlog du Sprint ..... 36

7.2 Intégration du module de paiement sécurisé ..... 36

7.3 Système d'évaluation et de feedback ..... 36

7.4 Dashboard Administrateur ..... 36

7.5 Conclusion du Sprint ..... 36


---

# Chapitre 7

# Réalisation - Sprint 4 : Paiement, Notation et Administration

## 7.1 Introduction et Backlog du Sprint

Cette section est en cours de rédaction.

## 7.2 Intégration du module de paiement sécurisé (D17 / Carte Bancaire)

Cette section est en cours de rédaction.

## 7.3 Système d'évaluation et de feedback (Calcul de la note moyenne chauffeur)

Cette section est en cours de rédaction.

## 7.4 Dashboard Administrateur (Gestion des comptes et validation des documents KYC)

Cette section est en cours de rédaction.

## 7.5 Conclusion du Sprint

Cette section est en cours de rédaction.

# Conclusion générale

Cette section est en cours de rédaction.

# Bibliographie et Netographie

Cette section est en cours de rédaction.

# Annexes

Cette section est en cours de rédaction.